

Section 1.2: Logistic Regression

AI Class — Lecture Notes [Student Copy]

1 From Regression to Classification

In Section 1.1 we predicted a *continuous* output $y \in \mathbb{R}$. Now suppose the output is a **binary label**: $y \in \{0, 1\}$. For example, we might ask:

Input x	Label y
Tumor size (cm)	0 = benign, 1 = malignant
Hours studied	0 = fail, 1 = pass
Email features	0 = not spam, 1 = spam

Why not use linear regression? If we fit $\hat{y} = wx + b$ directly and threshold at 0.5, the model is not constrained: it can predict values far below 0 or above 1, which are meaningless as probabilities and sensitive to outliers. What we really want is a model that outputs a *probability*:

$$\hat{p} = P(y = 1 \mid x) \in (0, 1).$$

2 The Sigmoid Function: Motivation via Log-Odds

A natural way to build a probability model is to assume that the **log-odds** (logit) of the event is a linear function of the input.

The **odds** of an event with probability p are defined as:

$$\text{odds} = \frac{p}{1 - p}.$$

We assume the **log-odds** are linear:

$$\log \frac{p}{1 - p} = wx + b. \tag{1}$$

This is a reasonable model: the log-odds can range over all of $\mathbb{R}$, which matches a linear function perfectly.

2.1 Solving for p

Exponentiating both sides of (1) and solving for p :

$$\begin{aligned} \frac{p}{1 - p} &= e^{wx+b} \\ p &= (1 - p) e^{wx+b} \\ p(1 + e^{wx+b}) &= e^{wx+b} \end{aligned}$$

$$\hat{p} = \underline{\hspace{10em}}$$

This function $\sigma(z) = \frac{1}{1 + e^{-z}}$ is called the **sigmoid** (or logistic) function.

Key properties of $\sigma(z)$.

- Output is always in $(0, 1)$ — valid as a probability.
- $\sigma(0) = 0.5$: the decision boundary is where $w x + b = 0$.
- As $z \rightarrow +\infty$, $\sigma(z) \rightarrow 1$; as $z \rightarrow -\infty$, $\sigma(z) \rightarrow 0$.
- Its derivative has the elegant form $\sigma'(z) = \sigma(z)(1 - \sigma(z))$ (see Appendix A).

3 The Logistic Regression Model

For d features, the model generalises directly:

$$\hat{p}^{(i)} = \sigma(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}) = \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}^{(i)}}}, \quad (2)$$

where $\boldsymbol{\theta} = [b, w_1, \dots, w_d]^\top$ and $\mathbf{x}^{(i)}$ includes a leading 1 for the bias (same design-matrix convention as Section 1.1).

The predicted class is:

$$\hat{y}^{(i)} = \begin{cases} 1 & \text{if } \hat{p}^{(i)} \geq 0.5 \\ 0 & \text{otherwise.} \end{cases}$$

4 Cost Function: Cross-Entropy via MLE

4.1 Setting Up the Likelihood

We model each label $y^{(i)}$ as a Bernoulli random variable with success probability $\hat{p}^{(i)}$:

$$P(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}) = (\hat{p}^{(i)})^{y^{(i)}} (1 - \hat{p}^{(i)})^{1-y^{(i)}}.$$

Assuming the n observations are independent, the joint likelihood is:

$$\mathcal{L}(\boldsymbol{\theta}) = \prod_{i=1}^n (\hat{p}^{(i)})^{y^{(i)}} (1 - \hat{p}^{(i)})^{1-y^{(i)}}.$$

4.2 From MLE to the Cross-Entropy Loss

Maximising $\mathcal{L}$ is equivalent to maximising the log-likelihood $\ell = \log \mathcal{L}$ (log is monotone):

$$\ell(\boldsymbol{\theta}) = \underline{\hspace{10em}}$$

Minimising the **negative** log-likelihood (and averaging over n) gives the **Binary Cross-Entropy (BCE)** loss:

$$J(\boldsymbol{\theta}) = \underline{\hspace{10cm}}$$

Why cross-entropy? When $y^{(i)} = 1$, only the first term $-\log \hat{p}^{(i)}$ matters: we are penalised heavily if $\hat{p}^{(i)}$ is close to 0 (confident and wrong). When $y^{(i)} = 0$, only the second term $-\log(1 - \hat{p}^{(i)})$ matters. The two terms switch on and off depending on the true label.

5 The Gradient and Why We Need Gradient Descent

Unlike MSE in linear regression, there is **no closed-form solution** for the minimiser of $J(\boldsymbol{\theta})$. The cross-entropy loss is convex (it has a unique global minimum), but we cannot invert the normal equations here because the sigmoid is non-linear.

Taking the gradient with respect to $\boldsymbol{\theta}$ (using the chain rule and the identity $\sigma'(z) = \sigma(z)(1 - \sigma(z))$):

$$\nabla_{\boldsymbol{\theta}} J = \underline{\hspace{10cm}}$$

Remarkably clean form. The gradient has exactly the same structure as in linear regression — it is a weighted sum of the input vectors, where the weights are the *prediction errors* $(\hat{p}^{(i)} - y^{(i)})$. The sigmoid “absorbed” all the complexity through the chain rule.

Because there is no closed form, we use **gradient descent**: starting from an initial $\boldsymbol{\theta}^{(0)}$, we repeatedly update

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J,$$

where $\alpha > 0$ is the **learning rate**. Gradient descent will be covered in detail in the next section.

6 Summary

Model	$\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$
Loss function	BCE: $J = -\frac{1}{n} \sum [y \log \hat{p} + (1 - y) \log(1 - \hat{p})]$
Gradient	$\nabla_{\boldsymbol{\theta}} J = \frac{1}{n} \sum (\hat{p}^{(i)} - y^{(i)}) \mathbf{x}^{(i)}$
Solution method	Gradient descent (no closed form)
Works well when	The decision boundary is (approximately) linear

Up next (Section 1.3): What if no linear boundary can separate the classes? We revisit the XOR dataset to show that both linear regression and logistic regression fail —

motivating the need for non-linearity and neural networks.

A Derivative of the Sigmoid Function

Let $\sigma(z) = (1 + e^{-z})^{-1}$. By the chain rule:

$$\sigma'(z) = -\frac{-e^{-z}}{(1 + e^{-z})^2} = \frac{e^{-z}}{(1 + e^{-z})^2}.$$

Factor out $\frac{1}{1 + e^{-z}}$ from the numerator and denominator:

$$\sigma'(z) = \frac{1}{1 + e^{-z}} \cdot \frac{e^{-z}}{1 + e^{-z}} = \sigma(z) \cdot \frac{1 + e^{-z} - 1}{1 + e^{-z}} = \sigma(z)(1 - \sigma(z)).$$

$$\sigma'(z) = \sigma(z)(1 - \sigma(z))$$

This compact form makes the chain-rule calculations in gradient descent particularly clean.